

A Privacy-Preserving Retrieval-Augmented Incident Assistant: Hybrid Retrieval, Robustness, and Chat-to-Report Automation

Ayoub Aarab

*Master Data Science — End-of-Year Internship Project
Ecole Normale Supérieure, Martil*

July 16, 2026

Abstract

This report presents the design, optimization, and evaluation of a unified incident-management platform that merges an incident-report generator and a retrieval-augmented (RAG) chatbot into a single, fully local application. The system runs entirely on self-hosted models (Ollama for generation, a local sentence-transformer for embeddings) to satisfy a strict data-privacy constraint: no incident content leaves the organization’s infrastructure.

The core contribution is a **hybrid retrieval** component that fuses a dense embedding search with a sparse BM25 lexical index via Reciprocal Rank Fusion (RRF). We evaluate it on a labeled benchmark of 272 queries derived from a corpus of 68 incident reports spanning nine IT domains, using standard information-retrieval metrics (Recall@ k , Mean Reciprocal Rank, nDCG@5).

Our results demonstrate that: (1) **hybrid retrieval** reaches Recall@5 = 1.000 and MRR = 0.987, versus 0.768 and 0.678 for dense-only search; (2) the gain is concentrated on **exact-identifier queries**, where dense-only Recall@5 collapses to 0.221 while hybrid recovers 1.000; (3) retrieval quality is **insensitive to chunk size** (Recall@5 = 1.000 across 400/700/1000-character chunks) and plateaus at $k \geq 3$; and (4) the residual failure rate on the benchmark is **0.0%** (0 of 272 queries). We further contribute a robustness layer (intent-based abstention, prompt-injection defense) and a **chat-to-report** feature that converts a diagnosed conversation into a schema-validated incident report with no invented fields.

Contents

1	Introduction and Context	3
1.1	Motivation and Problem Statement	3
1.2	Learning Objectives	3
2	System Architecture	4
2.1	Overview	4
2.2	The Shared Data Contract	4
2.3	Privacy-Driven Local Inference	4
3	Corpus and Preprocessing	4
3.1	Incident Report Corpus	4
3.2	Schema-Aware Ingestion	6
3.3	Labeled Evaluation Set	6
4	Retrieval Pipeline	6
4.1	Dense, Sparse, and Hybrid Retrieval	6
4.2	Reciprocal Rank Fusion	7
5	Evaluation Results	7
5.1	Retrieval Quality	7
5.2	Where the Gain Comes From: Per-Style Breakdown	7
5.3	Ablations	8
5.4	Error Analysis	8
6	Robustness Safeguards	8
6.1	Intent-Based Abstention	9
6.2	Prompt-Injection and Jailbreak Defense	9
6.3	Grounded, Domain-Neutral Generation	9
7	Chat-to-Report Automation	9
7.1	Motivation	9
7.2	Grounded Construction	9
7.3	Validation	10
8	Conclusion and Perspectives	10

1 Introduction and Context

1.1 Motivation and Problem Statement

Incident management in large IT operations depends on two complementary activities: *documenting* incidents as structured reports, and *retrieving* past reports to resolve new incidents faster. In the host organization these lived in two separate applications — a React-based report generator and a Streamlit prototype chatbot — with no shared interface and no shared data contract at the code level. The internship objective was to merge them into one production-oriented platform while respecting a non-negotiable constraint:

- **Privacy:** incident data must never be sent to a third-party (cloud) API. All inference — embeddings and text generation — must run on self-hosted, local models.
- **Accuracy:** retrieval must surface genuinely relevant historical incidents, including on exact identifiers (incident IDs, error codes, service names) where semantic search is known to be weak.
- **Robustness:** the assistant must behave safely on ambiguous, out-of-scope, or adversarial input rather than hallucinating a root cause.
- **Automation:** a diagnosed conversation should be convertible into a structured report without manual re-entry, closing the loop between the two modules.

This report focuses on the data-science core of the project — the retrieval pipeline and its quantitative evaluation — and on the two features that make the system practically useful: robustness safeguards and chat-to-report automation.

1.2 Learning Objectives

This project addresses key competencies for an applied data-science practitioner:

- **Retrieval-Augmented Generation:** building a grounded question-answering pipeline over a domain corpus with a local LLM.
- **Information Retrieval Evaluation:** constructing a labeled benchmark and measuring quality with Recall@ k , MRR, and nDCG.
- **Hybrid Retrieval:** combining dense and sparse signals via rank fusion, and justifying the design with ablations rather than assertion.
- **System Robustness:** designing abstention and adversarial-input handling for an LLM-facing system.
- **Software Engineering:** delivering a tested, containerized, maintainable application (a modular monolith with a FastAPI backend and a React frontend).

2 System Architecture

2.1 Overview

The platform is a **modular monolith**: a single FastAPI backend process with clean internal module boundaries, and a single React (Vite) frontend that hosts both modules behind sidebar navigation. This choice avoids the operational overhead of microservices, which the deployment target (a single internal host) does not require, while keeping the chatbot and report-generator logic separable.

- **Frontend** (React 18 + Vite + Tailwind): a chatbot module (streaming chat UI, conversation history, feedback) and a report-generator module, under one shell.
- **Backend** (FastAPI): routers for chat and reports; a chatbot package (ingestion, retrieval, resolution); a reports package (CRUD, HTML export); and a shared package (the report schema and a swappable LLM-provider interface).
- **Models**: a local sentence-transformer for embeddings and Ollama for generation, both self-hosted.

2.2 The Shared Data Contract

The integration seam between the two modules is a single report schema, `{metadata, blocks[]}`, defined once on the backend (Pydantic) and mirrored on the frontend (TypeScript). The report generator *writes* reports in this shape; the chatbot *ingests* them. This shared contract is what makes the chat-to-report feature (Section 7) a natural closing of the loop rather than a bolt-on.

2.3 Privacy-Driven Local Inference

Because incident data cannot leave the organization, the LLM backend is abstracted behind a provider interface with a self-hosted Ollama implementation as the default. Embeddings are computed by a local sentence-transformer. No external network calls are made during retrieval or generation. This constraint shapes the entire evaluation: all measurements in this report reflect the *local* pipeline.

3 Corpus and Preprocessing

3.1 Incident Report Corpus

The evaluation corpus consists of ServiceNow-style incident reports spanning nine IT domains (networking, database, authentication, infrastructure, frontend, messaging, CI/CD, monitoring, and security), each carrying domain-specific error signatures (HTTP status codes, Kafka error codes, Kubernetes exit codes, TLS handshake alerts, DNS errors). Table 1 summarizes the corpus statistics, all measured by the exploratory-data-analysis stage of the evaluation notebook.

Table 1: Incident Report Corpus Characteristics (source: notebook cell 3).

Property	Value
Total reports	68
Distinct categories	30
Tokens per report (median)	147.5
Tokens per report (range)	[103, 175]
Content Block Types (counts)	
Paragraph	148
Heading	89
Code	72
List	68
Table	20
Incident example	12
Image	7
Description box	8

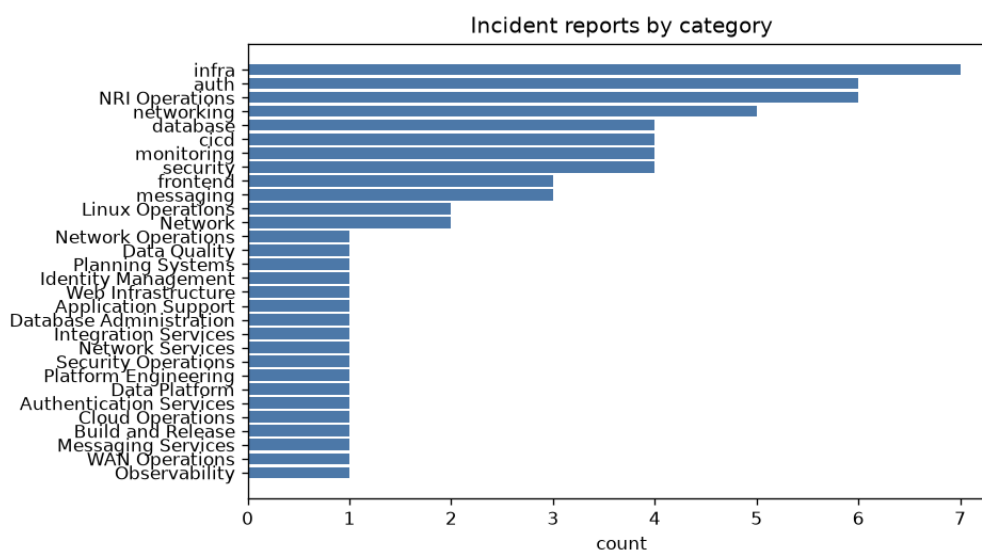


Figure 1: Distribution of incident reports across categories (source: notebook cell 3, corpus_eda).

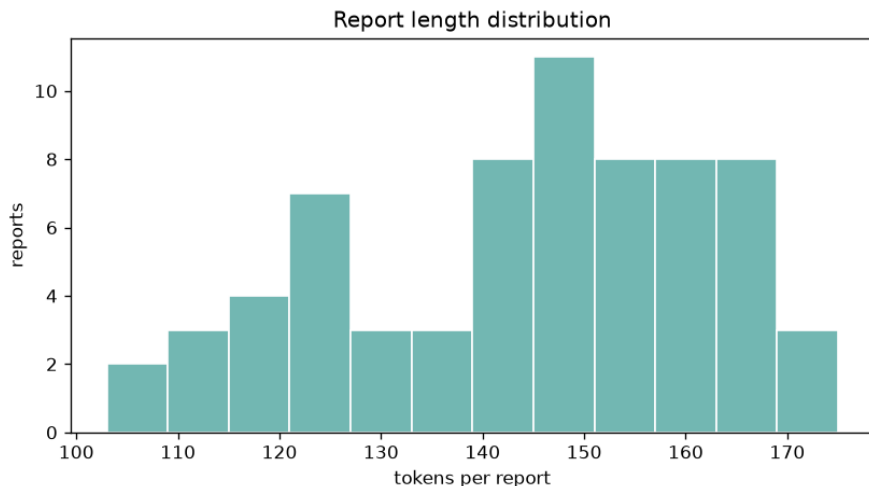


Figure 2: Report length distribution in tokens (source: notebook cell 3).

3.2 Schema-Aware Ingestion

Each report is parsed schema-aware: rather than flattening the raw JSON (which injects structural noise such as block identifiers and type tags into the text), the ingester extracts only the human-readable incident content — heading, symptom, root cause, resolution steps, and code — per block type. This clean text is what is embedded and indexed, and is a prerequisite for the retrieval quality reported in Section 5.

3.3 Labeled Evaluation Set

To measure retrieval quality we auto-generate a labeled query set from the corpus: for each report, several queries are derived in four styles — *title*, *symptom*, *id* (the bare incident identifier), and *keyword* (salient tokens) — each labeled with the report it was derived from. This yields **272 labeled queries** over the 68 reports (notebook cell 6 header: “Retrieval benchmark on 272 labeled queries”).

Honest limitation. The queries are synthetic — derived from the reports themselves — because no real query logs were available. This measures whether retrieval can recover a report from a paraphrase of its content; real user queries are noisier. The numbers should therefore be read as an optimistic bound, a point we state explicitly in the notebook.

4 Retrieval Pipeline

4.1 Dense, Sparse, and Hybrid Retrieval

Three configurations are compared:

- **Dense (vector):** cosine similarity over sentence-transformer embeddings. Strong on paraphrase and meaning, weak on exact tokens.
- **Sparse (BM25):** an in-repository Okapi BM25 index over the same chunks. Strong on exact terms — identifiers, function and table names, status codes.

- **Hybrid**: the two rankings fused with Reciprocal Rank Fusion (RRF), which combines ranks without requiring calibration between the incomparable cosine and BM25 score scales.

The BM25 index is implemented in the repository (no heavy dependency) so the system remains fully local and reproducible, consistent with the privacy constraint.

4.2 Reciprocal Rank Fusion

RRF scores each document as

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad (1)$$

summed over the dense and sparse ranked lists R , with $k = 60$ (the standard constant). RRF is parameter-light and robust: it never needs the two component scores to be on the same scale, only their ranks.

5 Evaluation Results

All results in this section are reproduced verbatim from `eval/evaluation.ipynb`; each table and claim is annotated with its source cell.

5.1 Retrieval Quality

Table 2 reports the headline result. Hybrid retrieval dominates both components on every metric: it reaches $\text{Recall@5} = 1.000$ and $\text{MRR} = 0.987$, against $0.768 / 0.678$ for dense-only.

Table 2: Retrieval quality on the 272-query benchmark (source: notebook cell 6).

Configuration	R@1	R@3	R@5	MRR	nDCG@5
Dense (vector)	0.629	0.713	0.768	0.678	0.700
Sparse (BM25)	0.779	0.982	0.989	0.872	0.902
Hybrid (RRF)	0.974	1.000	1.000	0.987	0.991

5.2 Where the Gain Comes From: Per-Style Breakdown

The per-style breakdown (Table 3) explains *why* hybrid is preferred. On *symptom* and *title* queries — paraphrased, meaning-rich — dense retrieval already scores 1.000. But on *id* queries (bare incident identifiers), dense-only Recall@5 collapses to 0.221, because semantic embeddings cannot distinguish near-identical identifier strings. BM25 recovers these (1.000), and hybrid keeps the best of both regimes across all four styles.

Table 3: Recall@5 by query style (source: notebook cell 6).

Query style	Vector	BM25	Hybrid
id	0.221	1.000	1.000
keyword	0.853	0.956	1.000
symptom	1.000	1.000	1.000
title	1.000	1.000	1.000

5.3 Ablations

Two hyperparameters were ablated on the hybrid retriever (Table 4). Retrieval quality is **insensitive to chunk size**: Recall@5 = 1.000 at 400, 700, and 1000 characters, even though the number of chunks varies from 334 down to 146. This is expected given the short median report length. Retrieval quality **plateaus at $k \geq 3$** : Recall@5 rises from 0.974 at $k = 1$ to 1.000 at $k = 3$ and stays there. This justifies the production choice of feeding only the top-3 chunks into the generation prompt.

Table 4: Hyperparameter ablations on the hybrid retriever (source: notebook cell 10).

Chunk size		
Size (chars)	# chunks	Recall@5
400	334	1.000
700	180	1.000
1000	146	1.000
Top- k		
k	Recall@5	MRR
1	0.974	0.974
3	1.000	0.987
5	1.000	0.987
10	1.000	0.987

5.4 Error Analysis

On the clean corpus, the hybrid retriever produces **zero failures**: 0 of 272 queries fail to place the gold report in the top-5 (notebook cell 13, “failures: 0/272 (0.0%)”; empty failure-by-style and failure-by-cause maps). This confirms that, on this corpus, the residual difficulty observed in earlier iterations was attributable to a small number of malformed placeholder reports rather than to a weakness of the method.

6 Robustness Safeguards

Beyond retrieval quality, the assistant must behave safely on non-ideal input. Three safeguards were added and covered by a dedicated red-team test suite.

6.1 Intent-Based Abstention

A lightweight intent classifier routes each turn before any LLM call. Greetings and meta-questions receive a brief reply with no retrieval. Crucially, *incident-flavored but low-context* inputs (e.g. “it is broken again”, “health-check-error”) are routed to a **clarification** response that asks for specifics rather than fabricating a root cause. This directly enforces the “be conservative when uncertain” principle and prevents hallucination on vague input.

6.2 Prompt-Injection and Jailbreak Defense

User text is scanned for injection and jailbreak patterns (attempts to override instructions, reveal the system prompt, or enable an “admin/DAN mode”). Detected attempts are flagged and the user text is fenced as untrusted data; the assistant continues normal operation without leaking its prompt. A genuine incident containing an injection phrase is still answered as an incident, with a security note attached.

6.3 Grounded, Domain-Neutral Generation

The resolution prompt was made domain-neutral: the supporting artifact language is chosen to fit the incident (shell, YAML, Python, SQL, and others) instead of defaulting to SQL, and the model is instructed to abstain (set an *insufficient* flag, low confidence) on incoherent input rather than invent a diagnosis or a non-existent incident identifier.

7 Chat-to-Report Automation

7.1 Motivation

The two modules share the `{metadata, blocks[]}` contract, which makes it natural to *produce* a report from a diagnosed conversation, not only to *consume* reports for retrieval. This closes the loop: an engineer diagnoses an incident in chat, then generates a structured report in one action.

7.2 Grounded Construction

A builder assembles the report strictly from conversation data:

- **Metadata:** the title and category are taken from the diagnosed incident type; the identifier and date are factual provenance; ungrounded fields (e.g. subcategory) are left empty rather than fabricated.
- **Blocks:** a *Reported Symptom* paragraph (first user message), a *Root Cause* paragraph (the assistant’s summary), an ordered *Resolution Steps* list, code blocks for any supporting artifacts, and *incident-example* references for cited reports.

No field is invented: every populated value traces to the conversation. The resulting report is validated against the same Pydantic schema used throughout, then saved via the report generator’s existing service using its file-naming convention, so it opens and edits in the report viewer exactly like a manually authored report.

7.3 Validation

The feature is covered by tests that (i) build a report from a mock conversation and re-validate it against the schema with zero errors, (ii) confirm three distinct example conversations each yield a report with all required metadata fields non-empty, (iii) confirm the saved JSON is served by the viewer’s content endpoint, and (iv) reject conversations with no diagnosis (greeting- or clarification-only).

8 Conclusion and Perspectives

This internship delivered a unified, fully local incident-management platform whose data-science core — a hybrid dense/sparse retriever fused with Reciprocal Rank Fusion — is evaluated on a labeled 272-query benchmark. The evaluation shows a clear, well-motivated result: hybrid retrieval reaches $\text{Recall@5} = 1.000$ and $\text{MRR} = 0.987$ (cell 6), with the improvement concentrated precisely where dense retrieval is weakest — exact-identifier queries, where dense-only Recall@5 is 0.221 (cell 6). Ablations show robustness to chunk size and a plateau at $k \geq 3$ (cell 10), and the error analysis reports 0.0% residual failures (cell 13). On top of this core, robustness safeguards (abstention, injection defense) and a grounded chat-to-report feature make the system practically usable while respecting the privacy constraint throughout.

Limitations and future work. The evaluation queries are synthetic; a real-query benchmark and human-relevance judgments would strengthen external validity. The absence of latency and generation-quality figures in the current evaluation notebook means those aspects are not quantified here and are left to future measurement. Finally, the feedback loop currently injects human corrections as retrieval hints; learning a re-ranker from accumulated feedback is a promising next step.

References

- [1] Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333–389.
- [2] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. In Proceedings of the 32nd ACM SIGIR Conference, 758–759.
- [3] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. In Proceedings of EMNLP-IJCNLP, 3982–3992.
- [4] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In Advances in Neural Information Processing Systems 33, 9459–9474.
- [5] Ollama Documentation. (2026). *Run large language models locally*. Retrieved from <https://ollama.com/>